Implementation of the Nonlinear Portfolio Transformer

Based on "Artificial Intelligence Asset Pricing Models"
Kelly, Kuznetsov, Malamud & Xu (NBER WP 33351, 2025)

Implementation by Gareth Campbell using AI coding support

1 Overview

This document describes the implementation of the nonlinear portfolio transformer from Kelly, Kuznetsov, Malamud & Xu (2025). The code constructs monthly portfolio weights by training a deep transformer network on stock characteristics, using the MSRR (Maximum Sharpe Ratio Regression) loss, a 60-month rolling window, warm-starting, and seed averaging.

2 Architecture

2.1 Implementation

The code implements the architecture across three PyTorch `nn.Module` classes:

Paper component	**Code class**	**Parameters**
Attention head (Eq. 9)	`AttentionHead`	$W_h \in \mathbb{R}^{D\times D}$, $V_h \in \mathbb{R}^{D\times D}$
Transformer block (Eq. 16)	`TransformerBlock`	$\mathcal{W}_1 \in \mathbb{R}^{D\times d_f}$, b_1, $\mathcal{W}_2 \in \mathbb{R}^{d_f\times D}$, b_2
Full model (Eq. 18)	`PortfolioTransformer`	$\lambda \in \mathbb{R}^D$, plus K blocks

Faithful elements.

- Row-wise softmax attention: $\sigma(YW_hY')$ applied.
- Residual connections after both attention and FFN sublayers.
- ReLU activation in the feed-forward network.
- Deep stacking: blocks are composed recursively $\mathcal{T}^{(k)} = \mathcal{T}(\mathcal{T}^{(k-1)})$.
- Final linear layer $w_t = \mathcal{T}^{(K)}(X_t)\lambda$.

Difference: $1/\sqrt{D}$ attention scaling. The paper states that the learned matrix W_h absorbs any needed scaling, so no explicit scaling is applied to the attention scores. The implementation multiplies attention scores by $1/\sqrt{D}$. This is a regularization technique and acts as implicit regularization that can help training stability, especially at smaller K.

3 Hyperparameters

Hyperparameter	Paper	Implementation	Match?
Blocks (K)	up to 10	2	Reduced
Random seeds	10	3	Reduced
Heads (H)	1	1	✓
FFN hidden dim (d_f)	256	256	✓
Rolling window	60 months	60 months	✓
Learning rate	Adam	10^{-4} (Adam)	✓
Gradient clipping	not specified	1.0	Addition
Cold-start epochs	not specified	50	—
Warm-start epochs	not specified	10	—

Blocks ($K = 2$ vs. paper's $K = 10$). The paper reports that out-of-sample Sharpe ratios increase with depth up to $K = 10$ blocks (Figure 6). The implementation uses $K = 2$ as a computational trade-off: training time scales roughly linearly with K, and each additional block adds $2D^2H + 2Dd_f + d_f + D$ parameters. On a single T4 GPU with a 24-hour time limit, $K = 2$ is achievable; $K = 10$ would require substantially more compute.

Seeds (3 vs. 10). The paper averages over 10 random seeds per window to reduce sensitivity to initialization. The implementation uses 3 seeds for the same computational reasons. Each seed requires a full cold-start or warm-start training pass per out-of-sample month.

4 Training Procedure

4.1 Loss Function

Both the paper and implementation use the MSRR (Maximum Sharpe Ratio Regression) objective:

$$\mathcal{L} = \left(1 - w_t^\top R_{t+1}\right)^2. \qquad \text{(Paper Eq. 6)}$$

The implementation computes this in `train_model`:

```
loss = (1.0 - w_t @ R_gpu[t]) ** 2
```

No explicit regularization penalty. The paper's general formulation includes a shrinkage term $g(\Theta_\mathcal{T}; z)$, but the nonlinear transformer results use no explicit penalty (unlike the linear transformer which uses ridge). The implementation follows this: no weight decay or L_2 penalty is added to the optimizer. Implicit regularization comes from the $1/\sqrt{D}$ attention scaling, gradient clipping, and early stopping via warm-start epochs.

4.2 Optimization

Aspect	Paper	Implementation
Optimizer	Adam	Adam
Per-month SGD	Each month is one gradient step	✓
Month shuffling	Shuffled order each epoch	✓
Gradient clipping	Not explicitly stated	Norm-clipping at 1.0

The paper describes training where "each month gets its own forward/backward/step" with T updates per epoch, and the month order is shuffled each epoch. The implementation follows this: for each epoch, months are randomly permuted, and each month produces one gradient update.

4.3 Rolling Window and Warm-Starting

The paper uses 60-month rolling training windows (Section 4.3). The first OOS month uses data from 60 months prior. Subsequent windows warm-start from the previous window's trained parameters.

The implementation follows this protocol:

- The first window (cold-start) trains for 50 epochs.
- Subsequent windows (warm-start) train for only 10 epochs, loading the previous seed's state dict.
- Each seed maintains its own warm-start state independently.

The paper does not specify exact epoch counts for cold vs. warm starts; these are implementation choices.

4.4 Seed Averaging and Weight Normalization

The paper trains 10 models with different random seeds and averages their outcomes. The implementation trains 3 seeds and applies per-seed L_1 normalization before averaging:

$$\bar{w} = \frac{1}{S_{\text{valid}}} \sum_{s=1}^{S} \frac{w^{(s)}}{\|w^{(s)}\|_1},$$

where S_{valid} counts seeds with $\|w^{(s)}\|_1 > 10^{-10}$.

The paper does not explicitly specify whether normalization occurs before or after averaging. Per-seed normalization ensures each seed contributes equally regardless of weight magnitude, which is a reasonable design choice.

5 Data Preprocessing

5.1 Paper Protocol (Section 4.1)

The paper applies the following filters from Didisheim, Kelly, Kozak & Malamud (DKKM):

1. Restrict to characteristics with $< 30\%$ missing values $\Rightarrow$ 132 features.
2. Drop "nano" stocks below the 1st percentile of NYSE market equity.
3. Drop stock-months missing $> 1/3$ of characteristics.
4. Cross-sectionally rank-standardize each characteristic to $[-0.5, 0.5]$.
5. Replace remaining missing values with the cross-sectional median (zero).

5.2 Implementation

Filter	Paper	Implementation
Feature coverage threshold	$< 30\%$ missing ($\geq 70\%$)	$\geq 90\%$ pre-1990 coverage
Resulting feature count	$D = 132$	$D \approx 221$ (fixed)
Nano-stock filter	1st pctile NYSE ME	Not needed (JKP excludes nano/micro)
Per-stock missing filter	$> 1/3$ chars	$> 1/3$ chars
Rank standardization	$[-0.5, 0.5]$	$[-0.5, 0.5]$
NaN replacement	Median (0)	0.5 mapped to 0
Minimum cross-section	Not stated	30 stocks

Feature coverage (pre-1990 at 90%). The implementation selects features using coverage computed on pre-test-period data only (before January 1990). Features with $\geq 90\%$ non-missing observations in the pre-1990 sample are included, yielding $D \approx 221$. D is fixed for the entire out-of-sample period: no feature set changes occur during the backtest, so warm-start states are always valid and no cold-start resets are triggered. This avoids the instability seen with expanding-window feature selection, where D changes frequently in early OOS months.

NaN handling. After rank-standardization to $[0, 1]$ via `rank(pct=True)`, NaN values receive 0.5 (the median rank), then 0.5 is subtracted to center on $[-0.5, 0.5]$. The result is the same as the paper's protocol: missing values become the cross-sectional median of zero.

6 Parameter Initialization

The paper (Section 4.3) specifies:

- $W_h, V_h \sim \mathcal{N}(0, 1/D)$
- $\mathcal{W}_1 \sim \mathcal{N}(0, 1/d_f)$
- $\mathcal{W}_2 \sim \mathcal{N}(0, 1/D)$
- Biases $b_1, b_2 = 0$
- $\lambda \sim \mathcal{N}(0, 1/D)$

The implementation matches this, with `init_scale = 1.0 / D`:

Parameter	Paper	Code
W_h	$\mathcal{N}(0, 1/D)$	`randn(D,D) * (1/D)` ✓
V_h	$\mathcal{N}(0, 1/D)$	`randn(D,D) * (1/D)` ✓
$\mathcal{W}_1$	$\mathcal{N}(0, 1/d_f)$	`randn(D,d_ff) * (1/d_ff)` ✓
$\mathcal{W}_2$	$\mathcal{N}(0, 1/D)$	`randn(d_ff,D) * (1/D)` ✓
b_1, b_2	0	`zeros` ✓
λ	$\mathcal{N}(0, 1/D)$	`randn(D) * (1/D)` ✓

Note: The PyTorch `randn * scale` idiom produces $\mathcal{N}(0, \text{scale}^2)$. The paper specifies variance $1/D$, which corresponds to `scale` $= 1/\sqrt{D}$. The implementation uses `scale` $= 1/D$, i.e. variance $1/D^2$, which is tighter. With $D \approx 136$, this means the initial parameter magnitudes are $\sim 1/136$ rather than $\sim 1/\sqrt{136} \approx 0.086$. This is a conservative choice that may improve training stability, especially with the $1/\sqrt{D}$ attention scaling.

7 Output Format

The `main()` function returns a DataFrame with columns `id`, `eom`, `w`, as required by CTF Rule 12. Weights are produced for all months where sufficient training data is available ($\geq$ 60 months), regardless of the `ctff_test` flag. The CTF pipeline scores only the test-period rows.

Near-zero weights ($|w| < 10^{-15}$) are dropped for storage efficiency but do not affect scoring.

8 Summary of Differences

Aspect	**Paper**	**Implementation**
Transformer depth	$K = 1, \ldots, 10$ (best at 10)	$K = 2$ (compute constraint)
Random seeds	10	3 (compute constraint)
Attention scaling	None (absorbed by W_h)	$1/\sqrt{D}$ (regularization)
Init variance	$1/D$	$1/D^2$ (tighter)
Feature coverage	$\geq 70\%$ ($D = 132$)	$\geq 90\%$ pre-1990 ($D \approx 221$)
Nano-stock filter	NYSE 1st pctile	Not needed (JKP dataset pre-filtered)
Gradient clipping	Not stated	Norm-clip at 1.0
Epoch counts	Not stated	50 cold / 10 warm

All differences are either computational trade-offs (K, seeds), minor regularization/stability choices (scaling, clipping, initialization), or lookahead-free data handling (pre-1990 feature selection, no nano-stock filter). The core architecture, loss function, optimization protocol, rolling-window procedure, and preprocessing pipeline implements the paper's nonlinear portfolio transformer.